L6. SQL – Operații cu mulțimi

Operațiile cu mulțimi din SQL sunt derivate din cele din algebră. Pentru o pereche de relații (R1,R2) se definesc:

- Reuniunea (∪) - toate înregistrările din R1 și R2, fără duplicate. Este implementat de operatorul SQL **UNION**.
- Intersecția (∩) - înregistrările comune din R1 și R2. Este implementat de operatorul SQL **INTERSECT**.
- Diferența (/) - înregistrările care apar în R1 dar nu și în R2. Este implementat de operatorii SQL **MINUS** și **EXCEPT**.
- Produsul Cartezian (×) - toate înregistrările din R1 combinate cu toate înregistrările din R2. Este implementat de operația de **CROSS JOIN**.

Pentru a putea aplica operații cu mulțimi pe două relații, acestea trebuie să fie compatibile:

- Să conțină același număr de câmpuri
- Câmpurile corespondente să aibă tipuri identice sau compatibile.

Sintaxa generală folosită de operațiile cu mulțimi este:

SELECT r1.* **FROM** Relație1 r1 ...

[**UNION** [**ALL**]|**INTERSECT**|**MINUS**|**EXCEPT**]

SELECT r2.* **FROM** Relație2 r2 …

6.1. Reuniunea

Operatorul **UNION** combină rezultatele din două interogări într-un singur rezultat. În acest proces va elimina și duplicatele.

Dacă se dorește păstrarea răspunsurilor duplicate (care au corespondent în ambele interogări), se poate folosi operatorul **UNION ALL**.

Sintaxa generală este:

SELECT r1.* **FROM** Relație1 r1 ...

UNION [**ALL**]

SELECT r2.* **FROM** Relație2 r2 …

Reuniunea poate fi folosită pentru a implementa operația de **FULL OUTER JOIN** (ex. dacă aceasta nu este implementată de sistemul de gestiune de baze de date considerat, cum ar fi MySQL).

SELECT * **FROM** T1 **LEFT JOIN** T2 **ON** T1.id = T2.id

UNION

SELECT * **FROM** T1 **RIGHT JOIN** T2 **ON** T1.id = T2.id

În acest caz parametrul **ALL** este omis pentru a evita perechile duplicate rezulta în urma operației de **FULL JOIN**.



ACTIVITATE 1: Utilizând SQL Workshop -> SQL Commands rulați următoarea interogare SQL:

```
SELECT s1.sid,s1.nume,s1.an,s1.bursa FROM Student s1
  WHERE s1.bursa <750

UNION

SELECT s2.sid,s2.nume,s2.an,s2.bursa FROM Student s2
  WHERE s2.bursa >900
```

6.2. Intersecția

Operatorul **INTERSECT** va returna doar rândurile comune din rezultatele celor două interogări.

Sintaxa generală este:

```
SELECT r1.* FROM Relație1 r1 ...

    INTERSECT

SELECT r2.* FROM Relație2 r2 …
```



ACTIVITATE 2: Utilizând SQL Workshop -> SQL Commands rulați următoarele interogări SQL:

(Facultățile care au cursuri și luni și miercuri)

```
SELECT c1.fid FROM Curs c1WHERE c1.zi = 'Luni'

      INTERSECT

SELECT c2.fid FROM Curs c2WHERE c2.zi = 'Miercuri'
```

6.3. Diferența

Diferența este implementată în SQL de operatorii **MINUS** sau **EXCEPT**.

Obs. Unele baze de date relaționale nu au implementată operația de diferență cu mulțimi. În acest caz, se poate utiliza operatorul **NOT IN** împreună cu subinterogări.

Sintaxa generală este:

SELECT r1.* **FROM** Relație1 r1 ...

MINUS

SELECT r2.* **FROM** Relație2 r2 …



ACTIVITATE 3: Utilizând SQL Workshop -> SQL Commands rulați următoarele interogări SQL:

(studenții care au cursuri luni și nu au miercuri)

```
SELECT c.sid
    FROM Contract c
        JOIN ContractITems ci ON c.cno=ci.cno
        JOIN Curs cr ON cr.cid=ci.cid
    WHERE c1.zi = 'Luni'
MINUS
SELECT c.sid
    FROM Contract c
        JOIN ContractITems ci ON c.cno=ci.cno
        JOIN Curs cr ON cr.cid=ci.cid
    WHERE c1.zi = 'Miercuri'
```

6.4. ORDER BY și aliasuri în operații cu mulțimi

Pentru a pune un alias unei coloane, este de ajuns să fie asociat coloanei în prima interogare a operației cu mulțimi. Pentru a ordona rezultatul unei operații cu mulțimi, puneți o clauza ORDER BY la finalul interogării.

```
SELECT sid AS "ID Student" FROM Student
MINUS
SELECT sid FROM Contract
ORDER BY "ID Student"
```

6.5. Exerciții

Obs: în cazul în care nu se precizează câmpurile dorite în rezultat se vor afișa cele mai relevante câmpuri care permit identificarea înregistrării (ex. la Student: sid, nume și an).

Obs. La scrierea interogărilor bazate pe operații cu mulțimi se recomandă să se implementeze separat interogările care sunt folosite ca operatori și abia apoi să se folosească acestea în operația pe mulțimi. În felul acesta se pot controla mulțimile care vor furniza datele de intrare pentru operație și se poate face mai ușor depanarea separată a interogărilor care joacă rolul de operanzi.

L6.Ex1. Folosind reuniunea să se listeze toți studenții (sid, nume, anstud, bursa) care au bursa minimă împreună cu cei care au bursa maximă. Obs. Nu uitați, studenții care au bursa = 0 sau NULL se consideră că nu au bursă! (388 de rânduri returnate)

L6.Ex2. Folosind intersecția să se listeze facultățile (fid, nume, adr) care au cursuri care se țin și la etajul 1 și la etajul 7. (7 rânduri returnate)

L6.Ex3. Să se afișeze CID-ul cursurilor care nu au fost contractate niciodată folosind: (16 rânduri returnate pentru toate)

a) Folosind diferența - nu folosiți operații de join sau subinterogări!

b) Folosind doar subinterogări

c) Folosind doar joins.

L6.Ex4. Folosind operații cu mulțimi și subinterogări, să se listeze studenții care nu au bursa (0 sau NULL) și au ore în sala 'A111'.

L6.Ex5. Folosind operații cu mulțimi, să se listeze studenții (sid, nume, anstud, bursa) sunt înscriși facultatea care are numărul de telefon '0256-403211' și nu au ore cu profesorul 'Romina Clarisa Dinu'. (76 de rânduri returnate)